UNORDERED, STREAMING EXECUTION OF SPMD PROGRAMS

A DRAFT

Roman Saldygashev
sldg.roman@gmail.com

November 7, 2026

ABSTRACT

This is a draft article about an abstract machine and programming model for SPMD computation that addresses control-flow divergence through dynamic regrouping of kernel instances according to their execution state. Rather than preserving conventional execution ordering, the model sacrifices ordering guarantees to increase throughput by allowing instances following different control-flow paths to be dynamically reorganized into more coherent groups. The model is deliberately simplified and restricted to provide a foundation for exploring alternative mechanisms for scheduling and executing divergent SPMD workloads.

We implement the model as a virtual machine embedded in C++ and adapt it to x86 SIMD execution. The implementation combines SIMD warp-local execution and global regrouping. Evaluation on an AVX-512 processor shows that hybrid execution provides consistent performance improvements on irregular path-tracing workloads over a pure reconvergence-based SIMD backend, achieving speedup up to $1.54\times$ and $1.24\times$ on average.

1 Introduction

Single-program-multiple-data (SPMD) computation commonly involves executing a large number of independent program instances over a task set. Although individual instances may follow different control-flow paths, the size of the task set provides an opportunity to identify instances executing the same parts of the program. In particular, when a sufficiently large number of instances are active concurrently, there is a high probability that multiple instances share the same execution state and can therefore be grouped for parallel execution.

We explore an execution model that exploits this observation by dynamically regrouping SPMD instances according to their current control-flow state. Rather than requiring instances to remain in their original execution groups or preserving their relative execution order, the model organizes them according to the next operation they must execute. Instances following different paths can consequently progress independently and be regrouped whenever their execution states coincide. This approach treats control-flow divergence as a scheduling problem rather than an inherent limitation of the programming model.

We introduce a programming model and abstract machine based on this approach, and implement a prototype virtual machine with scalar and SIMD execution backends. The SIMD implementation combines warp-local execution with dynamic reconvergence and global regrouping, adapting the abstract execution model to fixed-width SIMD hardware. The prototype demonstrates that the model can be implemented across different execution backends and evaluates its behavior on representative computations. The resulting system therefore serves both as an experimental realization of the abstract machine and as a basis for future research into its compilation, scheduling, and hardware implementations. The complete prototype implementation is available as free software at `https://github.com/groundflyer/loom`.

2 Related Work

Divergent execution presents a recurring challenge for SIMD and SIMT architectures because concurrently executing instances may require different control-flow paths. Research has consequently explored a broad range of approaches

to managing divergence, from application-level reorganization and compiler transformations to dynamic scheduling, thread regrouping, and hardware reconvergence mechanisms.

2.1 Programmer- and Application-Level Coherence Extraction

A common approach to control-flow divergence is to extract coherence at the application level and explicitly arrange computation so that program pieces with similar execution behavior are processed together. This approach is particularly prevalent in ray tracing, where spatial coherence can be exploited to form more SIMD-efficient groups. Ray pocket reordering and traversal techniques sort or otherwise reorganize rays according to their expected traversal behavior [1, 2], while deferred and sorted shading similarly postpone computation and group shading work to improve execution coherence [3]. Wavefront path tracing takes this idea further by decomposing a monolithic path-tracing kernels into stages and maintaining separate work queues, allowing paths at the same stage to be processed together [4]. Related techniques for production CPU path-tracing include explicit extraction of local shading coherence and vectorized execution [5, 6].

Application-level scheduling is not limited to rendering. Irregular workloads can be similarly partitioned or batched according to their execution characteristics, including mixture-of-experts training and inference [7, 8], and Monte Carlo neutron transport [9].

2.2 Static Analysis, Vectorization, and Control-Flow Transformation

Static analyses have been developed to determine where and under what conditions SPMD instances diverge, including divergence analyses based on abstract interpretation [10, 11]. Compiler techniques have likewise addressed SPMD and SIMD vectorization through control-flow analysis and transformation [12, 13, 14, 15], including lightweight SPMD vectorization [16] and techniques for compiling SPMD kernels for CPUs [17, 18].

Control-flow transformations for data-parallel execution include conversion of control dependence to data dependence and predicated execution [19, 20]. Other approaches transform or restructure control flow to manage divergence [21, 22, 23, 24].

Partial evaluation and program specialization provide a broader compile-time mechanisms for deriving efficient implementations of high-performance programs [25, 26, 27].

2.3 Dynamic Scheduling and Regrouping

Dynamic scheduling has been widely used to manage irregular and fine-grained workloads. Task-based systems provide runtime scheduling and load balancing across heterogeneous processors [28]. GPU implementations have likewise explored centralized and distributed task queues, task stealing and donation, and specialized concurrent queues for persistent-thread workloads [29, 30, 31, 32]. Multi-level queues have also been used for irregular wavefront computations on heterogeneous CPU–GPU systems [33].

A related line of work dynamically reorganizes SPMD threads to improve SIMD/SIMT utilization. Dynamic vectorization systems form vectors from concurrently executing threads [34, 35]. Other approaches remap thread data or regroup threads according to their runtime control-flow behavior [36, 37, 38]. Warp regrouping can be applied selectively to limit reorganization overhead [39, 40], and branch or warp interweaving can execute distinct divergent paths concurrently [41].

2.4 Hardware Reconvergence and Warp Scheduling

Hardware support for divergent SIMT control flow initially relied on per-warp reconvergence stacks that serialize divergent paths and resume execution at reconvergence points. Dynamic warp formation instead regroups threads from divergent warps into new warps at runtime [42], while thread-block compaction extends this idea by forming compacted warps across a block [43]. Subsequent designs relaxed the single-path execution imposed by stack-based reconvergence [44, 45]. Other work modified the reconvergence stack and warp scheduler to expose additional scheduling opportunities [46].

Various techniques dynamically adjust SIMD organization or exploit otherwise inactive execution resources [47, 48, 49, 50]. Variable and hybrid warp-size organizations allow smaller groups of divergent threads to execute independently while retaining larger groups when beneficial for regular workloads [51, 52]. More recent approaches explore speculative reconvergence, where threads expected to execute common code are encouraged to converge before execution proceeds [53].

Shader Execution Reordering (SER) provides a hardware-supported mechanism for dynamically reorganizing shader invocations to improve execution coherence in real-time graphics applications [54, 55, 56].

3 Programming Model

We define a programming model in which the computation performed by individual kernel instances independently of their parallel execution.

In our model kernels are pure functions. They have no shared mutable state and cannot access shared arrays or the private state of other kernel instances. Each kernel's inputs and outputs are explicitly specified. All state required by an instance is therefore represented by input parameters and local variables. Uniform variables are restricted to constants.

The model is deliberately constrained by abstracting away shared memory and unbounded execution state. Isolation and fixed-size persistent state eliminates the need to define shared-memory consistency, synchronization and dynamically growing execution state within the model. These restrictions simplify the implementation of the execution model while leaving greater freedom in choice of scheduling, state representation, execution order, and other mechanisms, thereby providing a basis for experimentation with alternative execution strategies.

The sequential execution semantics of the model is largely inspired by Wadler's listless machine [57]. As in the listless machine, computation proceeds through a bounded persistent state. In the present model, this state is represented by the kernel's persistent virtual registers together with program counter.

3.1 Intermediate Representation

Kernels are expressed using an extensible assembly-like intermediate representation operating on virtual registers. The representation is designed to support unstructured control flow, in which a basic block may transfer control to multiple non-local successors. Virtual registers have no fixed architectural limit; the register set required by a kernel is determined by the programmer or compiler.

We divide instructions into two groups: computation and control flow. Computation instructions have one destination operand and at least one source operand. These includes data movement, conversion, arithmetic, etc. Control-flow instructions comprise unconditional jump and multi-way jump. Multi-way jumps provide a compact representation of unstructured control flow by selecting a successor from a set of labels using a runtime value, avoiding the need to represent each possible transition through separate conditional branches and, consequently, potentially reducing the number of basic blocks.

Ertl and Gregg [58] observed that indirect branches, which are fundamental to efficient implementations of interpreters using dynamically selected control-flow targets, were poorly supported by contemporary branch-prediction mechanisms and could account for a substantial fraction of execution time. These observations illustrate the performance consequences of treating dynamically selected control flow as uncommon case. In the present model, multi-way jumps are instead a first-class control-flow construct, allowing unstructured control flow to be represented directly in the intermediate representation.

Jumps can target labels only; indirect jumps to arbitrary addresses are not part of the programming model. Two labels have predefined meanings: `START` identifies the initial block of a kernel, and `HALT` terminates the execution.

The representation also provides helper computation instructions that allow speculative and predicated execution to be expressed explicitly. For example, `cmov` conditionally selects between values without introducing a control-flow split. Additional helper instructions are described in Section 5.1.

Instruction operands carry explicit type information. Types can be attached to individual register operands, as in `mov r0:i32 r1:u32`, where the instruction converts `u32` value from `r1` to `i32` in `r0`. Alternatively, an instruction can specify its operation type separately using LLVM-style notation, as in `add @f32 r2 r0 r1`.

For example, the following kernel computes the Fibonacci sequence for an input `n` and returns a single `u64` value:

```
%kernel fib n:u64
    le @u64 cmp n 1
    cmov @u64 c cmp n 1
    jmp cmp 0: work 1: HALT
work:
    mov @u64 p 0
    mov @u64 c 1
    mov @u64 i 2
```

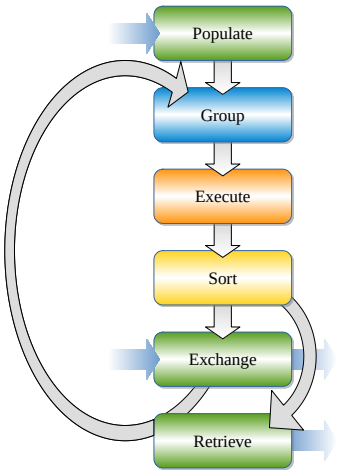


Figure 1: Execution flow of the abstract machine

```
loop:
    add @u64 t p c
    mov @u64 p c
    mov @u64 c t
    add @u64 i i 1
    le @u64 cmp i n
    jmp cmp 0: HALT 1: loop
%endkernel c:u64
```

4 Abstract Machine

The abstract machine defines the parallel execution of kernel instances described by the programming model. A fixed-capacity pool of persistent threads maintains the execution state of concurrently active instances, including their program counters and virtual registers.

4.1 Parallel Execution

Let N denote the input task-set size, C the thread-pool capacity, and P the number of processing elements (PE).

We first consider the case in which the input task set fits entirely in the thread pool, so that $N = C$ (Algorithm 1). We assume $P = C$, allowing all kernel instances to execute concurrently. Each input task is assigned to a persistent thread associated with a kernel instance. The `PC` of each thread identifies the basic block to be executed; in the present model, the `PC` is a pointer to a basic block.

The execution proceed by grouping threads according to their `PC`. Let G_{PC} denote a gang consisting of threads with the same `PC`. Each gang executes the operation with its common `PC` in parallel, after which the `PC`s of the threads may change. The threads are then sorted by `PC` and regrouped into gangs. This process continues until all kernel instances reach `HALT`. The resulting execution is predication-free within each gang: all threads in a gang execute the same operation while divergent control flow causes the threads to enter different gangs.

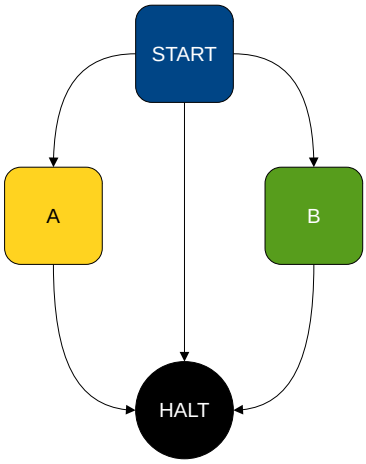
We next consider the general case $C \ll N$, where the input task set exceeds the capacity of the thread pool (Algorithm 2). Terminated kernel instances are removed from the pool and replaced by pending input tasks. We introduce an exchange operation that performs these replacements as a single operation: if D tasks remain pending and H pool entries contain terminated instances, then $E = \min(D, H)$ instances are exchanged. The operation can be viewed as simultaneous dequeue of terminated instances and enqueue of new tasks. This permits the pool to remain populated while the input task set is being consumed. Figure 1 presents a flowchart the described algorithm. Figure 2 illustrates the corresponding evolution of the thread-pool state for a kernel with unstructured control-flow.

Algorithm 1 Parallel execution with $N = C$

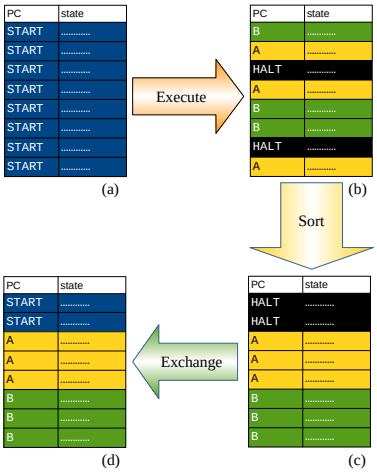
initialize persistent threads from the input task set
repeat
    Group threads into gangs G_{PC} by `PC`
    for all G_{PC} **do**
        Execute the operation identified by the `PC` of G_{PC} in parallel
    end for
    Sort threads by `PC`
until all threads have reached `HALT`

Algorithm 2 Parallel execution with a finite thread pool

Initialize persistent threads from the input task set
while $D > 0$ **do**
    Group threads into gangs G_{PC} by `PC`
    for all G_{PC} **do**
        Execute the operation identified by the `PC` of G_{PC} in parallel
    end for
    Sort threads by `PC`
    Exchange $E \leftarrow \min(D, H)$ terminated instances with pending tasks
end while
Execute Algorithm 1 on the remaining instances



(i) Control-flow graph with three basic blocks.



(ii) Thread-pool states during execution: initially populated pool (a), after an execution pass (b), after sorting (c), and after exchanging terminated instances with pending tasks (d).

Figure 2: Illustration of the abstract machine executing (ii) a kernel with unstructured control flow (i).

4.2 Execution Properties

The execution algorithms have several useful properties, assuming that kernel executions eventually terminate.

Queue-like execution Algorithm 2 naturally exhibits queue-like behavior: terminated instances leave the thread pool and are replaced by pending tasks from the input task set.

Starvation freedom Assuming that that the sorting algorithm is stable, both algorithms provide fair scheduling of persistent threads. Threads that remain active are not indefinitely bypassed by other threads, and consequently the execution is starvation-free.

Concurrent exchange Because the state of each thread is spatially isolated from the state of other threads, the exchange operation does not require serialized execution. In particular, replacement of terminated instances can be performed concurrently with execution of the current gangs. The exchange therefore need no introduce a sequential phase between successive execution rounds.

Unordered execution The principal drawback of these algorithms is the absence of ordering guarantees. In particular, although Algorithm 2 exhibits queue-like behavior, it does not provide FIFO semantics: the order in which kernel instances complete and are subsequently scheduled is determined by their execution state and the regrouping process. The model deliberately does not preserve execution order in exchange for throughput. Many computations do not require an ordering guarantee, while applications that do require one can maintain or reconstruct ordering information explicitly when the associated cost is sufficiently small relative to the computation throughput gained from unordered execution.

Fixed-pool-size invariant Both algorithms maintain a fixed pool size of C kernel instances. For Algorithm 1, this follows directly from the initial population and the absence of replacement during execution. For Algorithm 2, the same property is preserved by the exchange operation. If D tasks remain pending and H pool entries contain terminated instances, the exchange replaces $E = \min(D, H)$ instances, removing and inserting the same number of entries. Consequently, the pool continues to contain exactly C entries throughout execution.

When $D = 0$, any terminated instances that have not been exchanged simply remain in the pool until execution terminates. Thus the final retrieval operates on the complete pool and returns exactly C results. Consequently, the initial population, repeated exchanges, and final retrieval are sufficient to process the complete task set while maintaining a fixed pool capacity.

5 Implementation

Although the abstract machine provides a relatively simple model of parallel execution, implementing it efficiently on conventional hardware presents several challenges. In particular, the abstract machine assumes an arbitrary number of PEs and abstracts away costs such as state organization and regrouping, whereas practical processors impose fixed-width execution units, finite memory bandwidth, and non-negligible scheduling overhead. The prototype therefore adapts the abstract execution model to the capabilities and constraints of underlying hardware while preserving its execution properties.

The implementation is developed as a virtual machine embedded in C++. The following sections describe the implementation of the abstract machine.

5.1 Hybrid Execution on x86 SIMD

In the prototype implementation, the abstract machine is adapted to fixed-width SIMD execution model of x86 processors, targeting SIMD extensions from SSE through AVX-512.

x86 SIMD instructions operate on fixed-width architectural registers and therefore execute a fixed number of threads simultaneously. We consequently restrict the thread-pool capacity C to be divisible by the SIMD warp width W and organize the pool into C/W warps. A warp is therefore a fixed-width group of persistent threads. Sorting the thread pool does not, however, guarantee that each warp corresponds to a single gang: a gang may contain fewer than W threads, or a warp may contain threads belonging to multiple gangs. Such cases are handled through predication.

Reconvergence integration The principal implementation challenge is the cost of sorting. Even an efficient sorting algorithm introduces substantial overhead when sorting is performed after every execution pass, potentially eliminating

the benefit of regrouping threads. The prototype reduces the frequency of global sorting by allowing each warp execute multiple basic-block operations using predication and dynamic greedy-major reconvergence [59] adapted to the proposed programming model (Section 3). Warp execution is terminated when its utilization falls below a specified threshold U, at which point the warp is returned to the global scheduling and sorting process.

Threshold qualification The utilization threshold must be disabled for the final stages of execution to guarantee termination. Otherwise, a warp containing only a small number of active threads may repeatedly terminate early without making sufficient progress to complete those threads. The implementation therefore sets the utilization threshold to zero when the number of remaining active threads falls below a fixed constant Q. This constant can be derived analytically from the kernel's number of basic blocks B and the SIMD warp width W. In the prototype, the corresponding bound is determined by the cache-line capacity, i.e. the number of threads states that fit in a cache line, which also defines the minimum exchange granularity. Consequently, Q imposes a lower bound on the useful thread-pool capacity: choosing $C \leq Q$ makes the utilization-threshold optimization ineffective, since the threshold is disabled throughout execution.

Thread-Pool Capacity The choice of thread-pool capacity C affects the performance of the sorting and scheduling operations. Increasing C provides more capacity for concurrently executing kernel instances but also increases the cost of sorting the thread pool. Increasing C beyond $W(B+1)$ provides no additional benefit under the assumption that each basic block requires at least one warp to execute: the pool can already accommodate W threads for each basic block, plus one warp of terminated instances for exchange. However, this bound does not imply that $C = W(B+1)$ is optimal, since the likelihood of threads occupying different basic blocks depends on the workload's control-flow behavior. The optimal capacity may therefore depend on U and on the probability distribution of threads across basic blocks. Determining an appropriate capacity requires balancing the benefits of increased concurrency against the additional sorting overhead.

Sorting implementation The thread pool is sorted by program counter using counting sort. The program counter takes values from the finite set of basic-block identifies, making counting sort particularly well suited to the proposed execution model. In addition to producing the ordering required for regrouping threads into gangs, its counting phase determines the number of threads that have reached `HALT`. This information is reused whether terminated instances are available for exchange with pending tasks, avoiding a separate pass over the thread pool for this purpose.

HALT detection optimization The execution algorithms specify termination by testing whether all kernel instances have reached `HALT`. In the prototype, this test is reduced to a single comparison. Program counters are represented as unsigned integers, and `HALT` is assigned all-bits-set value, which is greater than every valid basic-block identifier. Consequently, after sorting, all terminated instances form a suffix of the thread pool. It is therefore sufficient to inspect the first thread in the pool: if its program counter is `HALT`, all remaining threads have also terminated.

SIMD-oriented IR instructions The IR instruction set was designed so that each instruction has a direct SIMD counterpart, allowing the same instruction sequence to be executed across the lanes of a SIMD register. This requires kernels targeting SIMD execution to operate on data types of uniform width. To simplify programming under this restriction, kernels are parameterized by a single *major* type, from which the type of the program counter is inferred. This restriction is sufficient for a broad class of numerical computations.

Several instructions are provided specifically to reduce control-flow overhead. In addition to `cmov`, the IR provides `slt` (select), which generalizes `cmov` by selecting a value from a set of registers using an index, and `sct` (select constant), which performs the corresponding operation on a set of constant values. These instructions allow control flow to be expressed as data selection where appropriate. Conditional move and conditional jump are implemented as SIMD blend operations. `slt` is implemented as a sequence of comparisons and blends, while multi-way jumps and `sct` map to SIMD shuffle and permute operations.

5.2 C++ Implementation

The prototype is implemented as a virtual machine embedded in C++, with several execution backends. The backends share the same programming model while providing progressively more specialized execution mechanisms.

The scalar backend, Basic Scalar Evaluator (BSE), provides a scalar reference implementation used primarily for validation. The Basic Vector Evaluator (BVE) implements the complete abstract-machine execution model in scalar fashion while operating on array-based thread pool. It establishes the structure-of-arrays (SoA) memory layout and the API for operating on the persistent thread pool described by Algorithms 1 and 2. The compact SIMD backend implements execution of a single warp. Finally, the hybrid SIMD backend implements the hybrid execution algorithm described in Section 5.1, integrating the compact SIMD backend with global thread-pool scheduling and regrouping.

The prototype exploits C++ template metaprogramming to minimize runtime overhead. All IR entities are encoded as class templates, and kernels are represented as type lists. Consequently, preprocessing stages including macro expansion, basic-block construction, adjacent-label merging, and various consistency checks are performed entirely during compilation of C++ source code.

Counting sort is partially vectorized and is particularly efficient on AVX-512 processors, where gather and scatter instructions can be used to accelerate data movement. Because counting sort is not performed in place, the implementation maintains two copies of the thread pool and alternates between them after each sorting operation.

The compact SIMD backend uses threaded block execution [58] by default. This execution strategy fits naturally with dynamic greedy-major reconvergence, since the next `PC` is selected directly from the set of successors of the currently executing block according to the major-group selection policy. This approach consistently outperforms central-loop-dispatch in the prototype.

The prototype also supports multi-processor execution. This extension follows directly from the spatial isolation of thread pools and the concurrent exchange property (Section 4.2). Each processor maintains its own thread pool, while the pools are coordinated centrally through exchange operation. Opportunistic try-lock-based management is used to coordinate access, requiring one mutex and one atomic flag per thread pool. The atomic flag indicates whether the thread pool is available for exchange.

5.3 Validation

The implementation was validated by comparing the different VM backends with one another and with those of reference implementations of the same kernels written in conventional C++. The reference kernels were independently tested for algorithmic correctness and used as the baseline for evaluating the VM implementations. The same kernel computations were then executed using BSE, BVE, compact SIMD, and hybrid SIMD backends, and their results were compared with each other and with those of the corresponding reference implementations.

6 Results

The prototype currently provides only an assembly-level programming interface, which makes developing benchmark kernels substantially more involved than using a conventional high-level programming model. Consequently, the available benchmark suite is relatively small. Nevertheless, it contains both simple validation kernels and substantially more complex path-tracing workloads, providing a basis for evaluating the hybrid execution model.

The first group consists of three kernels originally developed for validation of the VM: factorial, Fibonacci, and numerical root finding by bisection. These kernels provide relatively small and regular workloads. The second group consists of two variations of a smallpt-based [60] path tracer developed specifically for benchmarking the VM. Both path tracers implement adaptive pixel sampling and render a Cornell Box scene containing quads, triangles, disks, a cylinder, and a sphere. Path tracer v1 uses one importance-sampled diffuse material, while path tracer v2 additionally implements several different materials and probabilistic next-event estimation. Both programs have compact control flow and make extensive use of unstructured control flow by reusing code across different execution paths. For example, the same pseudo-random number generation routines are used along scene ray-traversal paths.

The complexity of the kernels varies substantially. The root-finding kernel consists of four basic blocks and 67 instructions, whereas path tracer v1 contains 44 basic blocks and 852 instructions, and path tracer v2 contains 54 basic blocks and 1423 instructions.

To increase the range of path-tracing workloads, both path tracers were evaluated using combinations of three adaptive-sampling thresholds (0.005, 0.001, and 0.0001), three pixel-sampling blur value (1, 5, and 20), and three maximum bounce depths (2, 5, and 20). This produces 27 path-tracing configurations. The resulting benchmark suite was used to compare the hybrid SIMD backend with the compact SIMD backend.

The comparison isolates the effect of global scheduling and regrouping. Both backends generate identical SIMD code for warp execution; the hybrid SIMD backend additionally performs global scheduling and regrouping (Section 5.1). The comparison therefore measures the benefit of the proposed hybrid execution strategy relative to executing the same warp-level SIMD code without global scheduling.

Experiments were performed on an AMD Zen 4 processor with AVX-512 support, Ryzen 7 7840HS, and on older AVX processor, Intel Core i7 2670QM. The simple validation kernels show no performance improvement from hybrid execution and mostly incur a performance loss relative to compact SIMD execution. In contrast, the path-tracing workloads consistently benefit from hybrid execution. On Ryzen 7840HS, the hybrid SIMD backend achieves speedups

of up to 1.54× and an average speedup of 1.24× across the path-tracing configurations. On Core i7 2670QM, the corresponding speedups are up to 1.24× and 1.11× on average.

7 Discussion and Future Work

An important direction for future work is the development of efficient techniques for partitioning SPMD programs into execution regions. The current implementation uses basic blocks as scheduling units, which provides a simple representation of control flow but can result in excessively fine-grained execution on modern SIMD architectures. Efficient execution regions should instead encompass larger portions of the control-flow graph, allowing conventional predicated execution and hardware-supported reconvergence to handle internal control flow without repeatedly returning to global scheduling.

Region formation must also account for the lifetime of the program state. In the current model, all virtual registers are persistent and retain their values across scheduling operations. This increases the size of the thread state and consequently the cost of regrouping and sorting. Partitioning programs into execution regions provides an opportunity to distinguish persistent values, which must survive transitions between regions, from temporary values whose lifetimes are confined to a single region. Such a distinction could substantially reduce the amount of state maintained for each kernel instance.

Hyperblocks provide one possible approach to constructing execution regions. By converting internal control flow into predicated execution, they allow multiple basic blocks to be executed as a single region while retaining the ability to represent divergent paths. Other region-formation techniques may provide different trade-offs between region size, predication overhead, register lifetime, and reconvergence opportunities. Developing such techniques is therefore an important prerequisite for exploiting the proposed execution model efficiently on modern SIMD and SIMT architectures.

Another promising direction is hierarchical regrouping of kernel instances. Modern GPUs organize their execution resources hierarchically, with multiple warps or wavefronts resident on each streaming multiprocessor (SM) or compute unit (CU), supported by local storage and scheduling resources. Rather than immediately returning partially utilized warps to a global scheduler, instances could first be regrouped locally within an SM or CU. This would allow execution resources to exchange instances and restore coherence without incurring the overhead of device-wide regrouping. Only when local regrouping can no longer maintain sufficient utilization would instances be returned to a global scheduling mechanism, where they could be redistributed across execution units. Such an approach would introduce multiple levels of scheduling granularity, with local regrouping exploiting execution-unit locality and global regrouping balancing work across the device.

Hierarchical regrouping should also be considered in conjunction with hardware latency-hiding mechanisms. Asynchronous memory transfers and overlapping computation with data movement can allow execution to proceed while other instances wait for memory operations to complete. Similarly, concurrent kernel execution can provide additional opportunities to occupy otherwise idle execution resources. Integrating these mechanisms with hierarchical scheduling could improve overall throughput by coordinating thread regrouping, data movement, and computation rather than treating them as independent operations.

References

[1] Solomon Boulos, Ingo Wald, and Carsten Benthin. Adaptive ray packet reordering. pages 131 – 138, 09 2008. ISBN 978-1-4244-2741-3. doi: 10.1109/RT.2008.4634633.

[2] Kirill Garanzha and Charles Loop. Fast ray sorting and breadth-first packet traversal for gpu ray tracing. *Comput. Graph. Forum*, 29:289–298, 05 2010. doi: 10.1111/j.1467-8659.2009.01598.x.

[3] Christian Eisenacher, Gregory Nichols, Andrew Selle, and Brent Burley. Sorted deferred shading for production path tracing. In *Proceedings of the Eurographics Symposium on Rendering*, EGSR '13, page 125–132, Goslar, DEU, 2013. Eurographics Association. doi: 10.1111/cgf.12158. URL `https://doi.org/10.1111/cgf.12158`.

[4] Samuli Laine, Tero Karras, and Timo Aila. Megakernels considered harmful: wavefront path tracing on gpus. In *Proceedings of the 5th High-Performance Graphics Conference*, HPG '13, page 137–143, New York, NY, USA, 2013. Association for Computing Machinery. ISBN 9781450321358. doi: 10.1145/2492045.2492060. URL `https://doi.org/10.1145/2492045.2492060`.

[5] Attila T. Áfra, Carsten Benthin, Ingo Wald, and Jacob Munkberg. Local shading coherence extraction for simd-efficient path tracing on cpus. In *Proceedings of High Performance Graphics*, HPG '16, page 119–128, Goslar, DEU, 2016. Eurographics Association. ISBN 9783038680086.

[6] Mark Lee, Brian Green, Feng Xie, and Eric Tabellion. Vectorized production path tracing. In *Proceedings of High Performance Graphics*, HPG '17, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450351010. doi: 10.1145/3105762.3105768. URL https://doi.org/10.1145/3105762.3105768.

[7] Yinghan Li, Yifei Li, Jiejing Zhang, Bujiao Chen, Xiaotong Chen, Lian Duan, Yejun Jin, Zheng Li, Xuanyu Liu, Haoyu Wang, Wente Wang, Yajie Wang, Jiacheng Yang, Peiyang Zhang, Laiwen Zheng, and Wenyuan Yu. Static batching of irregular workloads on gpus: Framework and application to efficient moe model inference, 2025. URL https://arxiv.org/abs/2501.16103.

[8] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. Megablocks: Efficient sparse training with mixture-of-experts, 2022. URL https://arxiv.org/abs/2211.15841.

[9] Braxton Cuneo and Mike Bailey. Divergence reduction in monte carlo neutron transport with on-gpu asynchronous scheduling. *ACM Trans. Model. Comput. Simul.*, 34(1), January 2024. ISSN 1049-3301. doi: 10.1145/3626957. URL https://doi.org/10.1145/3626957.

[10] Bruno Coutinho, Diogo Sampaio, Fernando Magno Quintao Pereira, and Wagner Meira Jr. Divergence analysis and optimizations. In *Proceedings of the 2011 International Conference on Parallel Architectures and Compilation Techniques*, PACT '11, page 320–329, USA, 2011. IEEE Computer Society. ISBN 9780769545660. doi: 10.1109/PACT.2011.63. URL https://doi.org/10.1109/PACT.2011.63.

[11] Julian Rosemann, Simon Moll, and Sebastian Hack. An abstract interpretation for spmd divergence on reducible control flow graphs. *Proc. ACM Program. Lang.*, 5(POPL), January 2021. doi: 10.1145/3434312. URL https://doi.org/10.1145/3434312.

[12] Ralf Karrenberg. *Automatic SIMD Vectorization of SSA-based Control Flow Graphs*. PhD thesis, 7 2014.

[13] Ralf Karrenberg and Sebastian Hack. Whole-function vectorization. In *Proceedings of the 9th Annual IEEE/ACM International Symposium on Code Generation and Optimization*, CGO '11, page 141–150, USA, 2011. IEEE Computer Society. ISBN 9781612843568.

[14] Vijay Kandiah, Daniel Lustig, Oreste Villa, David Nellans, and Nikos Hardavellas. Parsimony: Enabling simd/vector programming in standard compiler flows. In *Proceedings of the 21st ACM/IEEE International Symposium on Code Generation and Optimization*, CGO '23, page 186–198, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400701016. doi: 10.1145/3579990.3580019. URL https://doi.org/10.1145/3579990.3580019.

[15] Yuxuan Wang, Cristian Tirelli, Giovanni Ansaloni, Laura Pozzi, and David Atienza. An mlir-based compilation framework for control flow management on coarse grained reconfigurable arrays, 2026. URL https://arxiv.org/abs/2508.02167.

[16] Robin Kruppe, Julian Oppermann, Lukas Sommer, and Andreas Koch. Extending llvm for lightweight spmd vectorization: using simd and vector instructions easily from any language. In *Proceedings of the 2019 IEEE/ACM International Symposium on Code Generation and Optimization*, CGO 2019, page 278–279. IEEE Press, 2019. ISBN 9781728114361.

[17] Joachim Meyer, Aksel Alpay, Sebastian Hack, Holger Fröning, and Vincent Heuveline. Implementation techniques for spmd kernels on cpus. In *Proceedings of the 2023 International Workshop on OpenCL*, IWOCL '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400707452. doi: 10.1145/3585341.3585342. URL https://doi.org/10.1145/3585341.3585342.

[18] Ralf Karrenberg and Sebastian Hack. Improving performance of opencl on cpus. In *Proceedings of the 21st International Conference on Compiler Construction*, CC'12, page 1–20, Berlin, Heidelberg, 2012. Springer-Verlag. ISBN 9783642286513. doi: 10.1007/978-3-642-28652-0_1. URL https://doi.org/10.1007/978-3-642-28652-0_1.

[19] John R. Allen, Ken Kennedy, Carrie Porterfield, and Joe Warren. Conversion of control dependence to data dependence. In *Proceedings of the 10th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, POPL '83, page 177–189, New York, NY, USA, 1983. Association for Computing Machinery. ISBN 0897910907. doi: 10.1145/567067.567085. URL https://doi.org/10.1145/567067.567085.

[20] Aaron Smith, Ramadass Nagarajan, Karthikeyan Sankaralingam, Robert McDonald, Doug Burger, Stephen W. Keckler, and Kathryn S. McKinley. Dataflow predication. In *2006 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'06)*, pages 89–102, 2006. doi: 10.1109/MICRO.2006.17.

[21] Yunsup Lee, Ronny Krashinsky, Vinod Grover, Stephen W. Keckler, and Krste Asanović. Convergence and scalarization for data-parallel architectures. In *Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 1–11, 2013. doi: 10.1109/CGO.2013.6494995.

[22] Yunsup Lee, Vinod Grover, Ronny Krashinsky, Mark Stephenson, Stephen W. Keckler, and Krste Asanović. Exploring the design space of spmd divergence management on data-parallel architectures. In *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO-47, page 101–113, USA, 2014. IEEE Computer Society. ISBN 9781479969982. doi: 10.1109/MICRO.2014.48. URL https://doi.org/10.1109/MICRO.2014.48.

[23] Haicheng Wu, Gregory Diamos, Jin Wang, Si Li, and Sudhakar Yalamanchili. Characterization and transformation of unstructured control flow in bulk synchronous gpu applications. *The International Journal of High Performance Computing Applications*, 26(2):170–185, 2012. doi: 10.1177/1094342011434814. URL https://doi.org/10.1177/1094342011434814.

[24] Charitha Saumya, Kirshanthan Sundararajah, and Milind Kulkarni. Darm: Control-flow melding for simt thread divergence reduction – extended version, 2022. URL https://arxiv.org/abs/2107.05681.

[25] Roland Leißa, Klaas Boesche, Sebastian Hack, Arsène Pérard-Gayot, Richard Membarth, Philipp Slusallek, André Müller, and Bertil Schmidt. Anydsl: a partial evaluation framework for programming high-performance libraries. *Proc. ACM Program. Lang.*, 2(OOPSLA), October 2018. doi: 10.1145/3276489. URL https://doi.org/10.1145/3276489.

[26] Trevor L. McDonell. *Optimising Purely Functional GPU Programs*. PhD thesis, 2015.

[27] Trevor L. McDonell, Manuel M.T. Chakravarty, Gabriele Keller, and Ben Lippmeier. Optimising purely functional gpu programs. *SIGPLAN Not.*, 48(9):49–60, 9 2013. ISSN 0362-1340. doi: 10.1145/2544174.2500595. URL https://doi.org/10.1145/2544174.2500595.

[28] Tsung-Wei Huang, Dian-Lun Lin, Chun-Xun Lin, and Yibo Lin. Taskflow: A lightweight parallel and heterogeneous task graph computing system. *IEEE Transactions on Parallel and Distributed Systems*, 33(6):1303–1320, 2022. doi: 10.1109/TPDS.2021.3104255.

[29] Stanley Tzeng, Anjul Patney, and John D. Owens. Task Management for Irregular-Parallel Workloads on the GPU. In Michael Doggett, Samuli Laine, and Warren Hunt, editors, *High Performance Graphics*. The Eurographics Association, 2010. ISBN 978-3-905674-26-2. doi: 10.2312/EGGH/HPG10/029-037.

[30] David Troendle, Tuan Ta, and Byunghyun Jang. A specialized concurrent queue for scheduling irregular workloads on gpus. In *Proceedings of the 48th International Conference on Parallel Processing*, ICPP '19, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450362955. doi: 10.1145/3337821.3337837. URL https://doi.org/10.1145/3337821.3337837.

[31] Bernhard Kerbl, Michael Kenzel, Joerg H. Mueller, Dieter Schmalstieg, and Markus Steinberger. The broker queue: A fast, linearizable fifo queue for fine-granular work distribution on the gpu. In *Proceedings of the 2018 International Conference on Supercomputing*, ICS '18, page 76–85, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450357838. doi: 10.1145/3205289.3205291. URL https://doi.org/10.1145/3205289.3205291.

[32] Markus Steinberger, Michael Kenzel, Pedro Boechat, Bernhard Kerbl, Mark Dokter, and Dieter Schmalstieg. Whippletree: task-based scheduling of dynamic workloads on the gpu. *ACM Trans. Graph.*, 33(6), November 2014. ISSN 0730-0301. doi: 10.1145/2661229.2661250. URL https://doi.org/10.1145/2661229.2661250.

[33] George Teodoro, Tony Pan, Tahsin M. Kurc, Jun Kong, Lee A.D. Cooper, and Joel H. Saltz. Efficient irregular wavefront propagation algorithms on hybrid cpu–gpu machines. *Parallel Computing*, 39(4):189–211, 2013. ISSN 0167-8191. doi: https://doi.org/10.1016/j.parco.2013.03.001. URL https://www.sciencedirect.com/science/article/pii/S0167819113000343.

[34] Sajith Kalathingal, Sylvain Collange, Bharath N. Swamy, and André Seznec. Ditva: Dynamic inter-thread vectorization architecture. *Journal of Parallel and Distributed Computing*, 120:267–281, 2018. ISSN 0743-7315. doi: https://doi.org/10.1016/j.jpdc.2017.11.006. URL https://www.sciencedirect.com/science/article/pii/S0743731517303040.

[35] Teo Milanez, Caroline Collange, Fernando Magno Quintão Pereira, Wagner Meira, and Renato A. Ferreira. Thread scheduling and memory coalescing for dynamic vectorization of SPMD workloads. *Parallel Computing*, 40(9):548–558, October 2014. doi: 10.1016/j.parco.2014.03.006. URL https://inria.hal.science/hal-01087054.

[36] Eddy Z. Zhang, Yunlian Jiang, Ziyu Guo, Kai Tian, and Xipeng Shen. On-the-fly elimination of dynamic irregularities for gpu computing. In *Proceedings of the Sixteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS XVI, page 369–380, New York, NY, USA, 2011. Association for Computing Machinery. ISBN 9781450302661. doi: 10.1145/1950365.1950408.

[37] Eddy Z. Zhang, Yunlian Jiang, Ziyu Guo, and Xipeng Shen. Streamlining gpu applications on the fly: thread divergence elimination through runtime thread-data remapping. In *Proceedings of the 24th ACM International*

Conference on Supercomputing, ICS '10, page 115–126, New York, NY, USA, 2010. Association for Computing Machinery. ISBN 9781450300186. doi: 10.1145/1810085.1810104. URL https://doi.org/10.1145/1810085.1810104.

[38] Huanxin Lin and Cho-Li Wang. On-gpu thread-data remapping for nested branch divergence. *Journal of Parallel and Distributed Computing*, 139:75–86, 2020. ISSN 0743-7315. doi: https://doi.org/10.1016/j.jpdc.2020.02.003. URL https://www.sciencedirect.com/science/article/pii/S0743731518308967.

[39] Yaohua Yang, Shiqing Zhang, and Li Shen. Control divergence optimization through partial warp regrouping in gpgpus. In *Proceedings of the 2018 2nd International Conference on Computer Science and Artificial Intelligence*, CSAI '18, page 369–374, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450366069. doi: 10.1145/3297156.3297201. URL https://doi.org/10.1145/3297156.3297201.

[40] Yaohua Yang, Shiqing Zhang, and Li Shen. A lightweight method for handling control divergence in gpgpus. In *Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region*, HPCAsia '19, page 120–127, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450366328. doi: 10.1145/3293320.3293331. URL https://doi.org/10.1145/3293320.3293331.

[41] Nicolas Brunie, Caroline Collange, and Gregory Diamos. Simultaneous branch and warp interweaving for sustained gpu performance. *SIGARCH Comput. Archit. News*, 40(3):49–60, June 2012. ISSN 0163-5964. doi: 10.1145/2366231.2337166. URL https://doi.org/10.1145/2366231.2337166.

[42] Wilson Fung, Ivan Sham, George Yuan, and Tor Aamodt. Dynamic warp formation and scheduling for efficient gpu control flow. pages 407–420, 01 2008. ISBN 978-0-7695-3047-5. doi: 10.1109/MICRO.2007.30.

[43] Wilson W. L. Fung and Tor M. Aamodt. Thread block compaction for efficient simt control flow. In *2011 IEEE 17th International Symposium on High Performance Computer Architecture*, pages 25–36, 2011. doi: 10.1109/HPCA.2011.5749714.

[44] Minsoo Rhu and Mattan Erez. The dual-path execution model for efficient gpu control flow. In *2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*, pages 591–602, 2013. doi: 10.1109/HPCA.2013.6522352.

[45] Ahmed ElTantawy, Jessica Ma, Mike O'Connor, and Tor Aamodt. A scalable multi-path microarchitecture for efficient gpu control flow. pages 248–259, 02 2014. ISBN 978-1-4799-3097-5. doi: 10.1109/HPCA.2014.6835936.

[46] Licheng Yu, Xingsheng Tang, Minghui Wu, and Tianzhou Chen. Improving branch divergence performance on gpgpu with a new pdom stack and multi-level warp scheduling. *Journal of Systems Architecture*, 60(5):420–430, 2014. ISSN 1383-7621. doi: https://doi.org/10.1016/j.sysarc.2013.11.008. URL https://www.sciencedirect.com/science/article/pii/S1383762113002634. Embedded Systems Architecture and Applications.

[47] Jiayuan Meng, Jeremy W. Sheaffer, and Kevin Skadron. Robust simd: Dynamically adapted simd width and multi-threading depth. In *2012 IEEE 26th International Parallel and Distributed Processing Symposium*, pages 107–118, 2012. doi: 10.1109/IPDPS.2012.20.

[48] Jiayuan Meng, David Tarjan, and Kevin Skadron. Dynamic warp subdivision for integrated branch and memory divergence tolerance. In *Proceedings of the 37th Annual International Symposium on Computer Architecture*, ISCA '10, page 235–246, New York, NY, USA, 2010. Association for Computing Machinery. ISBN 9781450300537. doi: 10.1145/1815961.1815992. URL https://doi.org/10.1145/1815961.1815992.

[49] Ahmad Lashgar, Amirali Baniasadi, and Ahmad Khonsari. Dynamic warp resizing in high-performance simt, 2012. URL https://arxiv.org/abs/1208.2374.

[50] Ahmad Lashgar, Ahmad Khonsari, and Amirali Baniasadi. Harp: Harnessing inactive threads in many-core processors. *ACM Trans. Embed. Comput. Syst.*, 13(3s), March 2014. ISSN 1539-9087. doi: 10.1145/2567938. URL https://doi.org/10.1145/2567938.

[51] Timothy G. Rogers, Daniel R. Johnson, Mike O'Connor, and Stephen W. Keckler. A variable warp size architecture. *SIGARCH Comput. Archit. News*, 43(3S):489–501, June 2015. ISSN 0163-5964. doi: 10.1145/2872887.2750410. URL https://doi.org/10.1145/2872887.2750410.

[52] Xingxing Jin, Brian Daku, and Seok-Bum Ko. Improved gpu simd control flow efficiency via hybrid warp size mechanism. *Microprocessors and Microsystems*, 38(7):717–729, 2014. ISSN 0141-9331. doi: https://doi.org/10.1016/j.micpro.2014.06.007. URL https://www.sciencedirect.com/science/article/pii/S0141933114000982.

[53] Sana Damani, Daniel R. Johnson, Mark Stephenson, Stephen W. Keckler, Eddie Yan, Michael McKeown, and Olivier Giroux. Speculative reconvergence for improved simt efficiency. In *Proceedings of the 18th ACM/IEEE International Symposium on Code Generation and Optimization*, CGO '20, page 121–132, New York, NY, USA,

2020. Association for Computing Machinery. ISBN 9781450370479. doi: 10.1145/3368826.3377911. URL https://doi.org/10.1145/3368826.3377911.

[54] Microsoft Corporation. Directx raytracing (dxr) functional spec. URL https://microsoft.github.io/DirectX-Specs/d3d/Raytracing.html.

[55] NVIDIA Corporation. Nvidia optix 8.1 programming guide, 10 2024.

[56] Patrick Kelly, Yuriy O'Donnell, Kenzo Elst, Juan Cañada, and Evan Hart. *Ray Tracing in Fortnite*, pages 791–821. 08 2021. ISBN 978-1-4842-7184-1. doi: 10.1007/978-1-4842-7185-8_48.

[57] Philip Wadler. Listlessness is better than laziness: Lazy evaluation and garbage collection at compile time. In Robert S. Boyer, Edward S. Schneider, and Guy L. Steele Jr., editors, *Proceedings of the 1984 ACM Conference on LISP and Functional Programming, LFP*, pages 45–52. ACM, 8 1984. doi: 10.1145/800055.802020. URL https://doi.org/10.1145/800055.802020.

[58] M. Anton Ertl and David Gregg. The structure and performance of efficient interpreters. *Journal of Instruction-Level Parallelism*, 5, 11 2003.

[59] Xin Huo, Sriram Krishnamoorthy, and Gagan Agrawal. Efficient scheduling of recursive control flow on gpus. In *Proceedings of the 27th International ACM Conference on International Conference on Supercomputing*, ICS '13, page 409–420, New York, NY, USA, 2013. Association for Computing Machinery. ISBN 9781450321303. doi: 10.1145/2464996.2479870. URL https://doi.org/10.1145/2464996.2479870.

[60] Kevin Beason. smallpt: Global illumination in 99 lines of c++, 4 2007. URL http://www.kevinbeason.com/smallpt/.